



FIREBRAND

LYCEUM EDUCATION GROUP

Conditional Logic and Decision Structures

Python Pathway | Session 2

Session Recording & Professional Expectations

Session Recording

- All Estio training sessions are recorded to:
 - Support learner progress
 - Enable review of content
 - Maintain quality and compliance standards
- By attending this session, you acknowledge that recording is taking place.

Camera & Participation Expectations

- To support engagement and attendance monitoring:
 - Cameras should remain on during the session
 - Please participate actively in discussions and activities
 - If you are unable to use your camera or microphone for any reason, please inform your tutor directly via chat or message.

This session forms part of your professional apprenticeship experience. Please engage respectfully and professionally throughout.

Safeguarding, British Values & Wider Learning

Safeguarding protects individuals from harm, abuse, neglect, radicalisation and exploitation.

If you have a concern about yourself or someone else, email:

safeguarding@bpp.com

Or speak to your tutor privately.

We treat all concerns seriously and confidentially.

Safeguarding



The British Values are:

1. **Democracy**
2. **Rule of Law**
3. **Individual Liberty**
4. **Mutual Respect**
5. **Tolerance of Different Faiths and Beliefs**

These values underpin everything we do in professional and digital environments.

British Values



Prevent is about protecting people from being drawn into extremism or radicalisation.

In our programmes, Prevent links to:

- **Online safety**
- **Digital responsibility**
- **Critical thinking**
- **Professional ethics**

Prevent



As we go through today's session, consider:

How today's topic connects to British Values

Whether there are any Prevent or digital safety themes

How this learning supports your professional and career development

Your tutor will guide discussion and help you make these connections.

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, **K7**, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, **K9**, K11

Skills

S1, S4, S6, **S7**, S10,
S11, S12, **S16**

Behaviours

B2, B3

Lesson Aim and Objectives

Lesson Aim: Conditional Logic and Decision Structures

Lesson Objectives

Apply

- Apply conditional logic to control program behaviour

Analyse

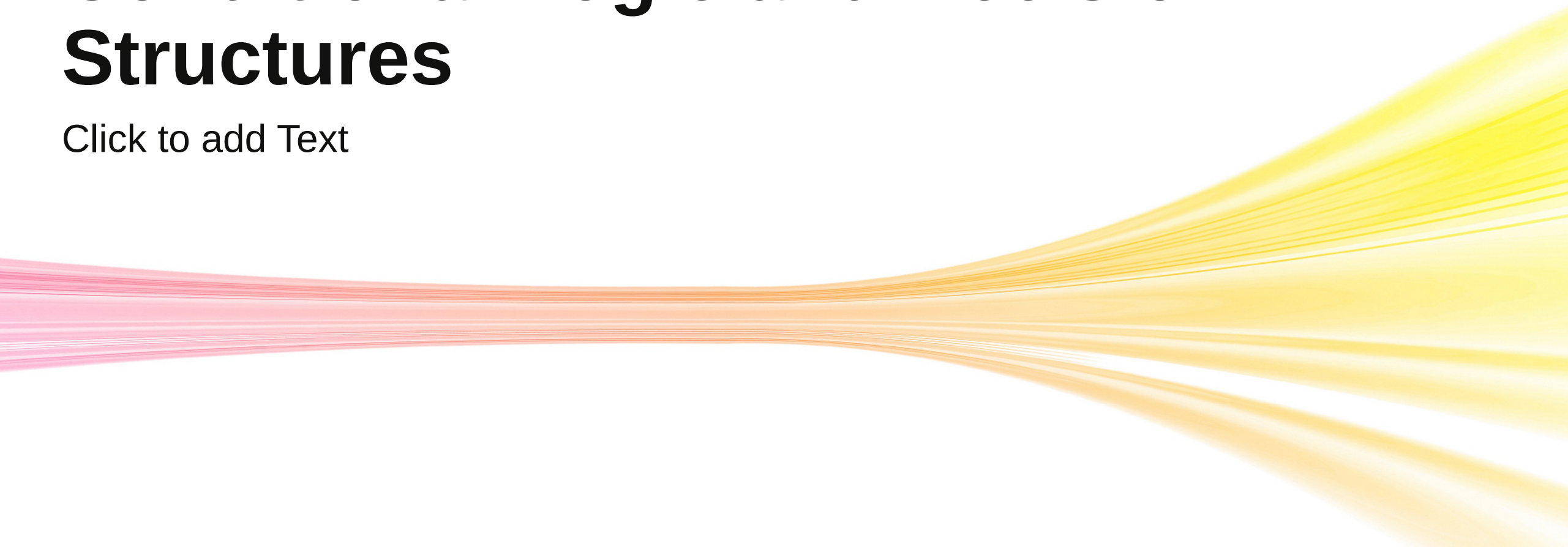
- Analyse decision structures using flowcharts and pseudocode

Evaluate

- Evaluate alternative logic approaches for clarity and maintainability

Conditional Logic and Decision Structures

Click to add Text

A decorative graphic consisting of several overlapping, wavy, horizontal lines that transition in color from pink on the left to orange and then yellow on the right. The lines are thin and numerous, creating a sense of motion and depth.

if, elif, else

- A condition evaluates to True or False (a **Boolean**)
- **if** runs a block only when its condition is True
- **elif** checks another condition if the first was False
- **else** catches everything remaining

Code Example:

```
age = 20
if age < 13:
    print("Child")
elif age < 18:
    print("Teenager")
else:
    print("Adult")
```

Boolean Operators

- **and** both sides must be True
- **or** at least one side must be True
- **not** flips True to False and vice-versa

Code Example:

```
age = 20
has_ticket = True
if age >= 18 and has_ticket:
    print("Entry allowed")
else:
    print("Entry denied")
```


From Real-World Rules to Code

The Business Rules

An online store wants to apply automated checkout discounts based on order value:

- **Rule 1:** Orders under £50 get no discount
- **Rule 2:** Orders between £50 and £100 get a 10% discount
- **Rule 3:** Premium orders over £100 get a 20% discount

The Programming Translation

- **Conditionals (if/elif/else)** act as the decision forks in our software
- **Expressions** evaluate to True or False to guide program flow
- **Order of Execution** determines which business rule takes priority

Pitfalls of Poor Conditional Ordering

The Danger of Overly Broad Nets

WARNING: This logic breaks business rules due to poor ordering!

```
discount = 0
```

```
if order_total >= 0:
```

```
    discount = 0
```

```
# Catches EVERYTHING first!
```

```
elif order_total >= 50:
```

```
    discount = 0.10
```

```
# Unreachable code
```

```
elif order_total >= 100:
```

```
    discount = 0.20
```

```
# Unreachable code
```


FIREBRAND
LYCEUM EDUCATION GROUP

discount = 0.00 # General safety net / default state

Design Principles for Code Maintainability



**SPECIFIC TO
GENERAL**



FAIL FAST



**READABILITY OVER
CLEVERNESS**

Designing Logic Before Coding

Map out decision paths as a flowchart or pseudocode before writing Python

This avoids missed cases and overly complex nested conditions

Convert each decision point directly into an if / elif / else statement

Right vs. Wrong (Conditional Logic)

Overly Complex / Wrong Order:

```
if age >= 0:  
    print("Valid")  
elif age >= 18:  
    print("Adult")  
# age>=18 branch is unreachable!
```

Corrected, Logically Ordered:

```
if age >= 18:  
    print("Adult")  
elif age >= 0:  
    print("Valid")  
else:  
    print("Invalid")
```

Why is the second condition in the flawed version never reached?

The second condition is never reached because **conditional structures evaluate from top to bottom and stop at the first true match**

Visual Modelling: Mapping Real-World Logic

The Mission

- Translate abstract business rules into concrete, visual blueprints using **flowcharts** or **pseudocode** before writing any Python code

Choose Your Scenario

Select **one** of the following real-world systems to model:

- **Login Validation:** Check if a username exists, verify the password, and handle account lockouts after 3 failed attempts
- **Discount Calculation:** Evaluate order totals to apply 0%, 10%, or 20% discounts accurately without overlapping ranges
- **Simple Eligibility Checker:** Determine if a user meets multi-variable criteria (e.g., age requirements combined with ticket ownership)

Your Tasks & Deliverables

Execution Steps

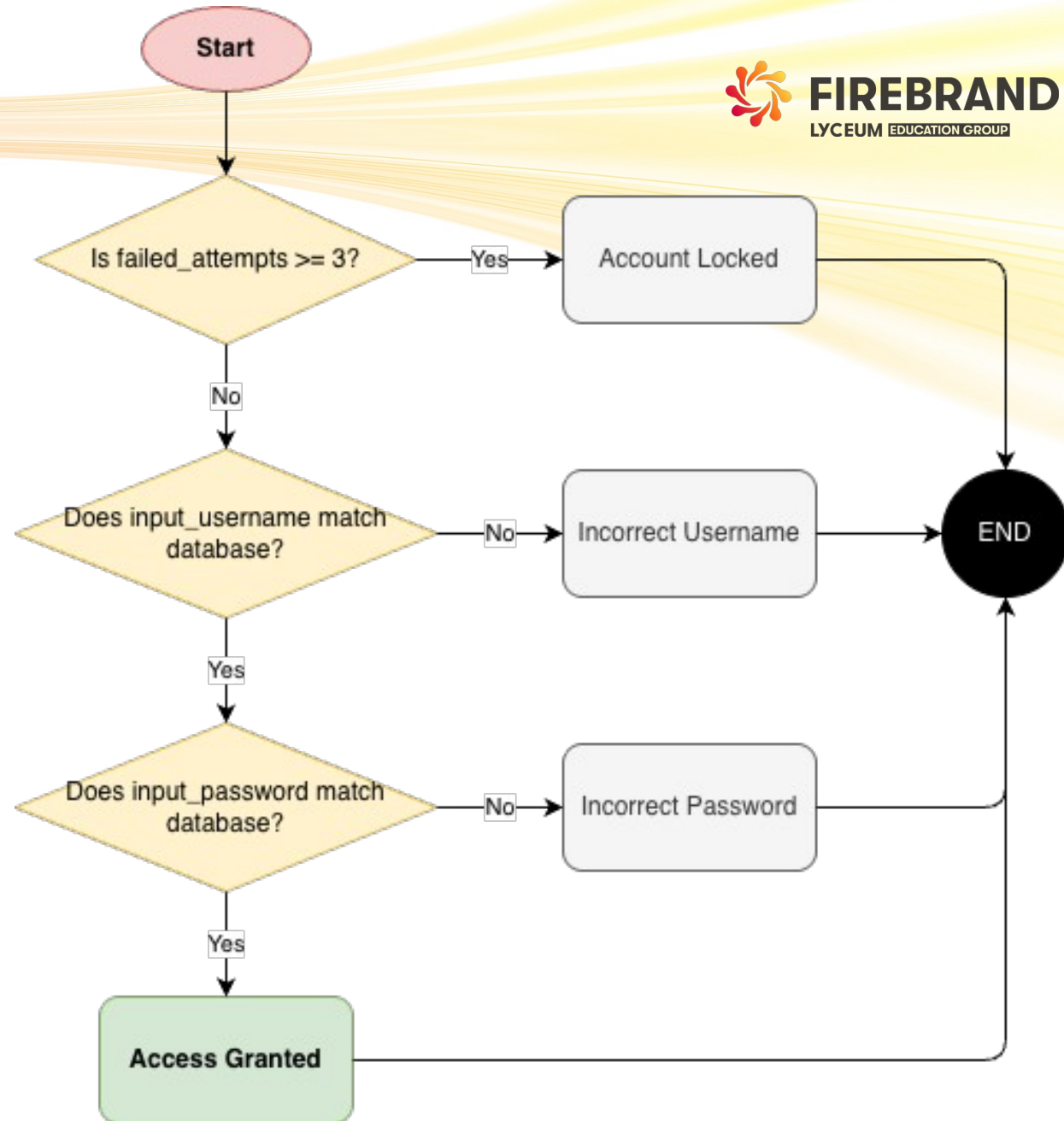
- **Draft the Model:** Sketch a visual flowchart (using diamonds for decisions) or write sequential pseudocode
- **Seal Every Leak:** Ensure all decision paths are complete, closed, and clearly defined (every "Yes" needs a matching "No")
- **Trace the Extremes:** Test your model with boundary values (e.g., exactly 0, exactly 18, or negative numbers) to catch hidden bugs

Expected Outcome

- A **clear, logical visual model** that fully represents the entire decision-making process
- A blueprint that a fellow programmer could easily read and convert into Python syntax without guessing

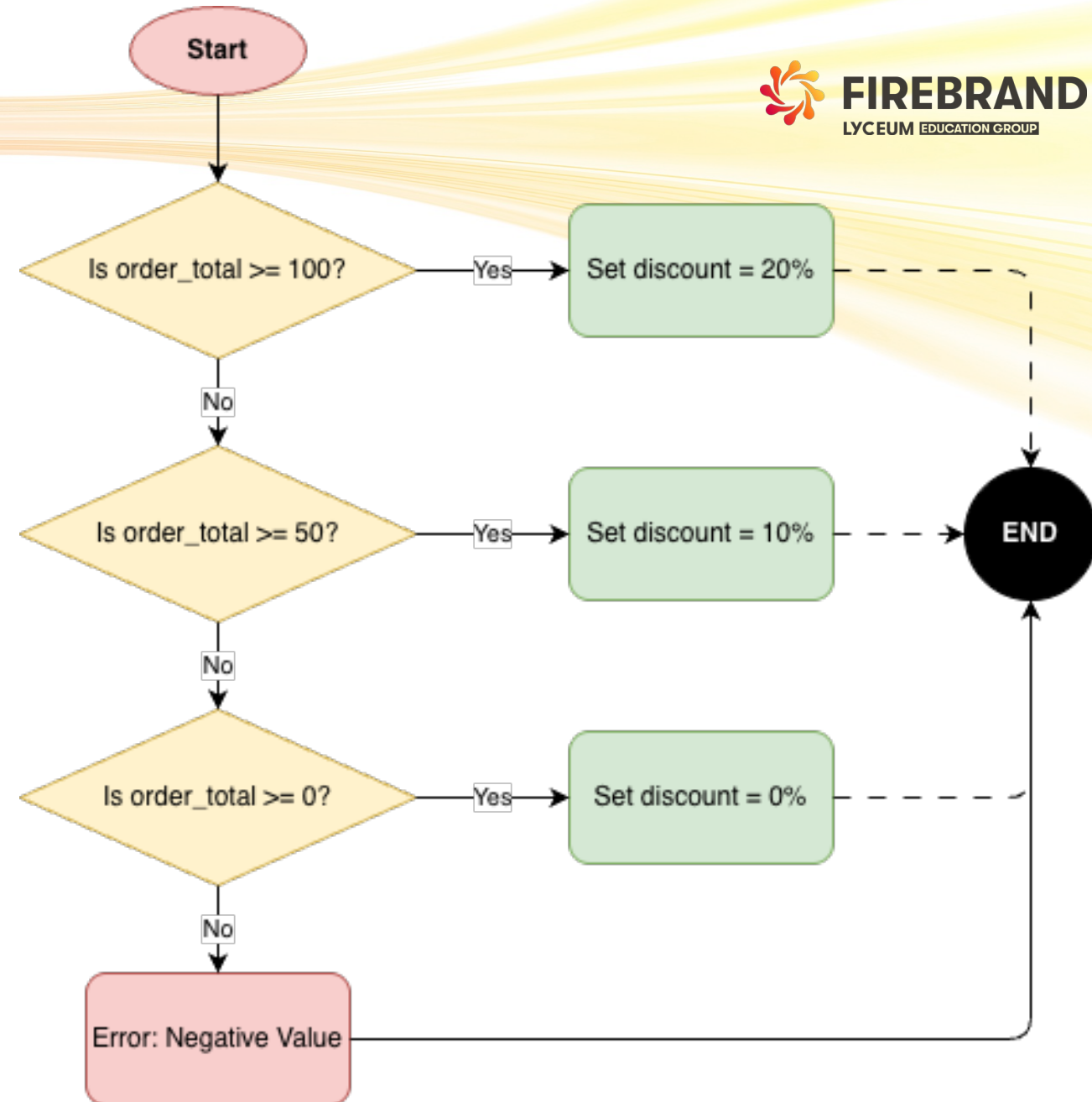
Login Validation

```
IF failed_attempts >= 3 THEN
    DISPLAY "Account Locked"
ELSE IF input_username != database_username THEN
    DISPLAY "Incorrect Username"
ELSE IF input_password != database_password THEN
    DISPLAY "Incorrect Password"
ELSE
    DISPLAY "Access Granted"
ENDIF
```



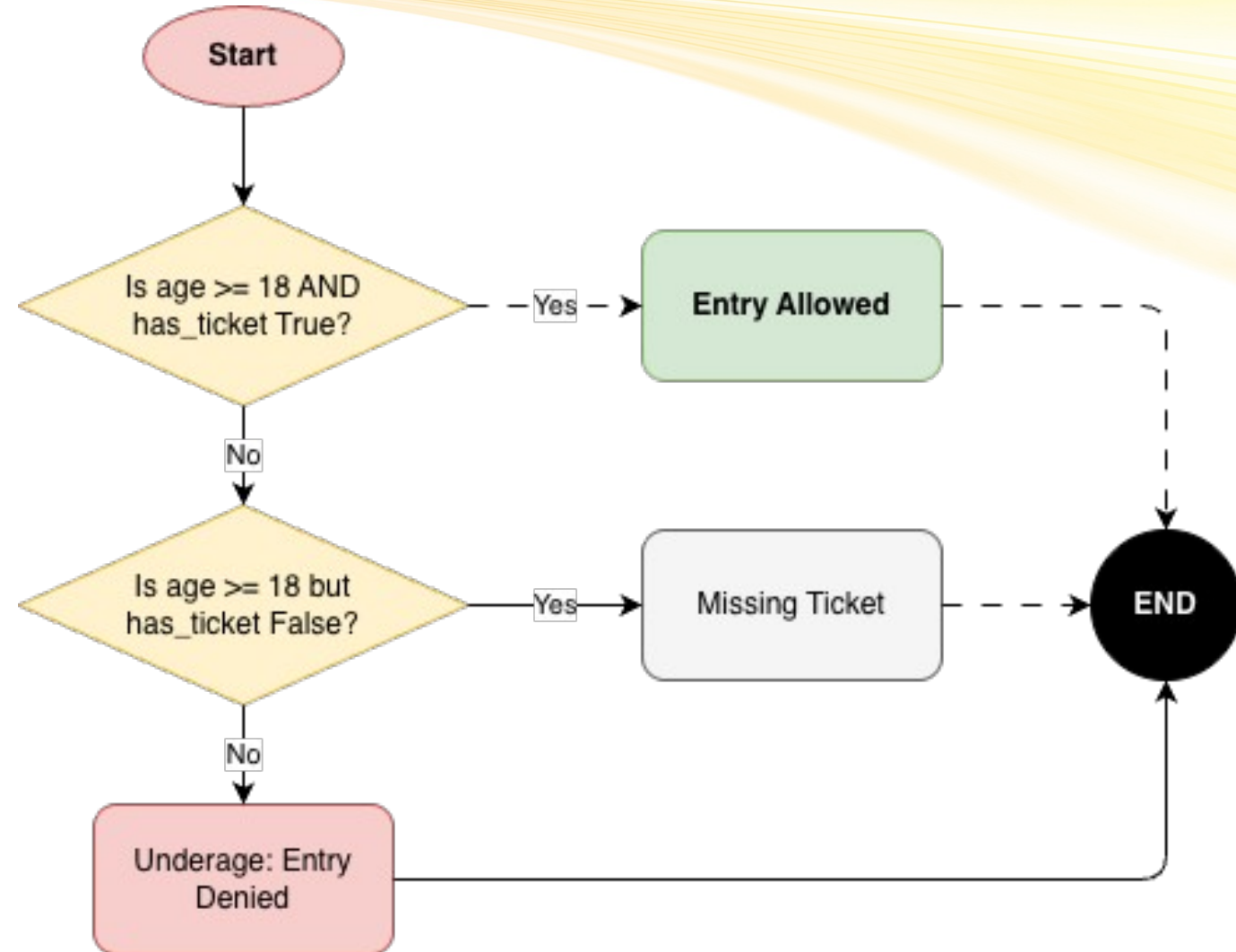
Discount Calculation

```
IF order_total >= 100 THEN
    discount = 20
ELSE IF order_total >= 50 THEN
    discount = 10
ELSE IF order_total >= 0 THEN
    discount = 0
ELSE
    DISPLAY "Error: Invalid Negative Total"
ENDIF
```



Simple Eligibility Checker

```
IF user_age >= 18 AND has_ticket == True THEN  
    DISPLAY "Entry Allowed"  
ELSE IF user_age >= 18 AND has_ticket == False  
THEN  
    DISPLAY "Missing Ticket"  
ELSE  
    DISPLAY "Underage: Entry Denied"  
ENDIF
```



Translating Logic into Code

The Core Strategy

- **Direct Mapping:** Convert flowchart shapes or pseudocode branches straight into if, elif, and else statements
- **Maintain Alignment:** Keep the code structure perfectly mirrored to your original design layout
- **Avoid Improvisation:** Write your script exactly as planned before optimising the logic

Testing and Refining Your Script

Now, you need to convert your flowchart or pseudocode into a working Python script:

Practical Code Review

- **Targeted Testing:** Run test variables through every condition path
- **Trace Outcomes:** Confirm that actual script execution matches intended flowchart paths
- **Bug Squashing:** Refine syntax gaps and eliminate any logic holes

The Ultimate Goal

- **Working Code:** Scripts must execute flawlessly without error
- **Accurate Logic:** Your program fully handles every predefined business condition

IT Helpdesk Priority System



ACTIVITY



Objective: Design and build a program that assigns a support ticket priority

Tasks / Requirements:

- Ask for urgency (High/Medium/Low) and user type (Staff/Student)
- Use if/elif/else to assign a priority level
- Print the assigned priority with a clear message

Key Challenge: Represent the logic as a flowchart before coding it

Extension Activity: Add a rule so Staff tickets are always raised by one priority level



ACTIVITY



The Next Activity will begin soon



Debugging and Refinement Activity



ACTIVITY



Common Issues Provided:

- Incorrect condition ordering
- Missing cases
- Overly complex nested logic

Tasks:

- Identify and annotate issues
- Simplify and restructure conditionals where appropriate
- Correct and re-run the script

Expected Outcome: Improved scripts demonstrating clear, efficient and correct conditional logic



ACTIVITY





FIREBRAND

LYCEUM EDUCATION GROUP

Evaluation Discussion

Architectural Trade-Offs

- **Structure:** Evaluate the visual clarity of deeply nested code versus flattened, simplified logic structures
- **Balance:** Weigh human readability against raw computational execution efficiency
- **Teamwork:** Analyse how your structural choices impact long-term code maintainability in collaborative team environments

Core Comparison Metrics

Readability, Efficiency, and Maintainability

- **Nested vs. Simplified:** Deeply nested if statements create cognitive load; flattening logic with logical operators makes code cleaner
- **Readability vs. Efficiency:** Computer processors handle both structures easily, but humans require clean layouts to prevent bugs
- **Team Impact:** Complex, undocumented logic paths slow down peer reviews and complicate future code updates

Expected Outcome

Key Takeaways

- **Clarity First:** Explain precisely how structural layout choices directly affect code comprehension
- **Performance Reality:** Recognise that design choices primarily optimize developer time rather than hardware performance
- **Sustainability:** Defend code design decisions based on how easily a teammate can maintain the script tomorrow

Debating Code Architecture

Discussion Questions

- **The Teammate Test:** If a new developer joined your project tomorrow, which of your debugging solutions would be easiest for them to understand without an explanation? Why?
- **The Scaling Challenge:** Imagine your code needs to handle **50 different conditions** instead of 3. Would a nested structure or a flat if/elif/else structure be easier to expand?
- **The "Clever" Trap:** Have you ever written a highly compact, "clever" line of code using logical operators that actually ended up being harder to read than a simple, long-form nested statement?

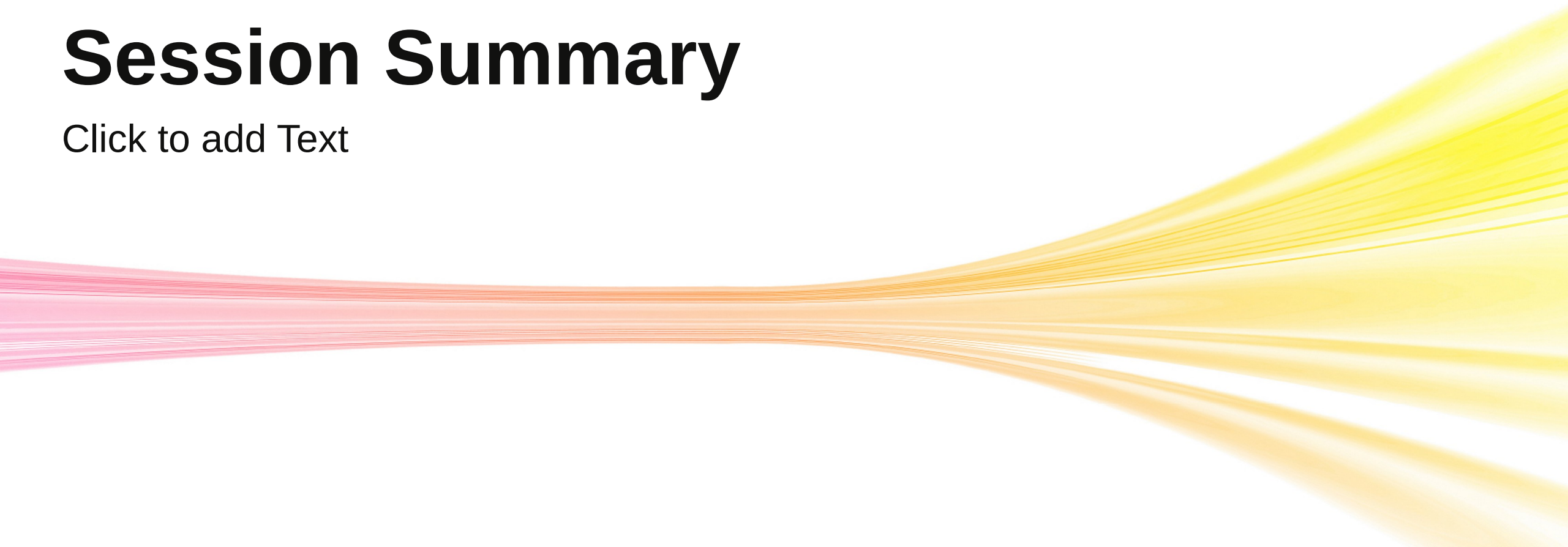
Stretch and Challenge

Improve your solutions by doing the following:

- Refactor the complex nested conditionals into simpler structures
- Combine conditions using logical operators
- Improve readability and reduce unnecessary complexity

Session Summary

Click to add Text



Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, **K7**, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, **K9**, K11

Skills

S1, S4, S6, **S7**, S10,
S11, S12, **S16**

Behaviours

B2, B3

Lesson Aim and Objectives

Lesson Aim: Conditional Logic and Decision Structures

Lesson Objectives

Apply

- Apply conditional logic to control program behaviour

Analyse

- Analyse decision structures using flowcharts and pseudocode

Evaluate

- Evaluate alternative logic approaches for clarity and maintainability

